

Assignment 2

Cricket Bash

Web Cricket Game

Course	CS-4032 Web Programming
Project Title	Cricket Bash — Web Cricket Game
Student Name	Muhammad Daniyal
Roll Number	i230579
Section	CS-F
Tech Stack	React (Vite), Zustand, Tailwind CSS, HTML5 Canvas

CS-4032 Web Programming

1. Introduction

Cricket Bash is a browser-based, single-innings batting simulation game developed as part of the CS-4032 Web Programming course. The project demonstrates modern front-end engineering practices by combining React component architecture, reactive state management, and HTML5 Canvas-based animations into a cohesive, interactive cricket experience.

The game challenges players to score as many runs as possible in a fixed innings while choosing between two distinct batting styles Aggressive and Defensive, each of which carries its own risk-reward trade-off. The power bar mechanic forces players to time their shots, translating a physical reflex into a probabilistic scoring outcome.

2. Tools and Technologies

The following technologies were employed in the development of Cricket Bash:

Technology	Purpose
React (Vite)	Component-based UI framework; Vite provides fast HMR and build tooling
Zustand	Lightweight global state management for game state (scores, phase, results)
Tailwind CSS	Utility-first CSS for rapid, consistent UI styling
HTML5 Canvas	Frame-by-frame rendering of bowling, power bar, and shot animations
requestAnimationFrame	Smooth, timer-accurate animation loop for all game phases

3. Game Logic and Working Mechanism

3.1 State Management (Zustand)

All game state is maintained in a centralized Zustand store, enabling any component to read or mutate state without prop drilling. Core state variables include:

- `battingStyle` — current player mode (Aggressive or Defensive)
- `runs`, `wickets`, `ballsBowled` — live match statistics
- `phase` — game lifecycle flag: `idle` | `bowling` | `slider` | `shot` | `gameover`
- `lastResult` — outcome of the most recent delivery (Wicket / 0–6)
- `commentary` — dynamically generated description of the last event

3.2 Ball-by-Ball Delivery Flow

Each delivery in Cricket Bash follows a deterministic seven-step sequence:

- Step 1 — Bowling animation starts on the Canvas; the ball travels from the bowler's end toward the batsman.
- Step 2 — On the ball reaching the batsman zone, the Power Bar activates and the slider begins oscillating.
- Step 3 — The player clicks the power bar to capture the slider position p (a value in the range 0.0 to 1.0).
- Step 4 — The captured position p is compared against the cumulative probability segments for the selected batting style.
- Step 5 — The segment containing p determines the outcome: Wicket, 0, 1, 2, 3, 4, or 6.
- Step 6 — The Zustand store is updated: runs/wickets increment, commentary is refreshed, and phase advances to shot.
- Step 7 — If the innings limit is reached, phase switches to gameover and the Game Over overlay is displayed.

3.3 Scoring Rules

- Numeric outcomes (1–6) add the corresponding number of runs to the total.
- A Wicket outcome increments the wicket counter without adding runs.
- The over display is formatted as overs.balls, derived from ballsBowled (e.g., 12 balls = 2.0 overs).

4. Probability Mapping to the Power Bar

4.1 Mechanism

The power bar is divided into coloured, contiguous segments. Each segment is proportional to its probability weight and is labelled with its outcome. When the player clicks, the current slider position p is compared against the ordered cumulative probability thresholds $c_1, c_2, c_3 \dots$ and the first threshold satisfying $p \leq c_i$ selects that segment's outcome.

4.2 Aggressive Style

Aggressive style maximises boundary potential at the cost of a substantially elevated wicket risk. The distribution is as follows:

Outcome	Probability
Wicket	35%
Dot Ball (0)	8%
1 Run	7%

Outcome	Probability
2 Runs	10%
3 Runs	5%
4 Runs (Boundary)	15%
6 Runs (Six)	20%

Cumulative breakpoints for aggressive style:

- 0.00 – 0.35 → Wicket
- 0.35 – 0.43 → Dot Ball (0)
- 0.43 – 0.50 → 1 Run
- 0.50 – 0.60 → 2 Runs
- 0.60 – 0.65 → 3 Runs
- 0.65 – 0.80 → 4 Runs
- 0.80 – 1.00 → 6 Runs

4.3 Defensive Style

Defensive style trades big-hitting potential for safety, reducing the wicket probability to 15% while concentrating mass in the 0–2 run range. The distribution is:

Outcome	Probability
Wicket	15%
Dot Ball (0)	20%
1 Run	25%
2 Runs	20%
3 Runs	8%
4 Runs (Boundary)	8%
6 Runs (Six)	4%

Defensive play is recommended for survival-focused batting or when wickets are at a premium.

5. Implementation of Animations

All animations are driven by `requestAnimationFrame`, which schedules redraws in sync with the display refresh rate. A time-delta (`dt`) parameter is passed to each animation callback to ensure

frame-rate-independent movement — the ball travels the same canvas distance per second regardless of device performance.

5.1 Bowling Animation

The ball is drawn at an initial position on the bowler's side of the canvas. Each frame, its x-coordinate is advanced by a velocity scaled to dt. When the ball reaches the batsman zone, the callback exits and the power bar phase begins.

5.2 Power Bar Animation

A vertical slider line oscillates horizontally between the left (0) and right (1) boundaries of the power bar. The direction flag reverses each time a boundary is reached, producing a continuous ping-pong motion. The current position is exposed to the click handler so the exact float value can be captured on player input.

5.3 Shot Animation

For all scoring outcomes (1, 2, 3, 4, and 6), the ball is projected from the batsman position along a run-dependent angle. Higher run values are assigned steeper launch angles to simulate genuine batting trajectories — a six, for example, arcs high before landing beyond the boundary, whereas a single travels low and short.

5.4 Visual Feedback

- Commentary text changes colour based on the outcome severity (e.g., red for a wicket, gold for a six).
- The batsman sprite is briefly highlighted for boundary and six outcomes to reward the player visually.
- The Game Over overlay is composited directly onto the canvas at innings completion.

6. Screenshot-Based Results

The following screenshots document the five required game states. Each screenshot includes a sticky note displaying the student's Name, Roll Number, and Section, as specified in the submission requirements.

Figure 1 — Aggressive Batting in Action

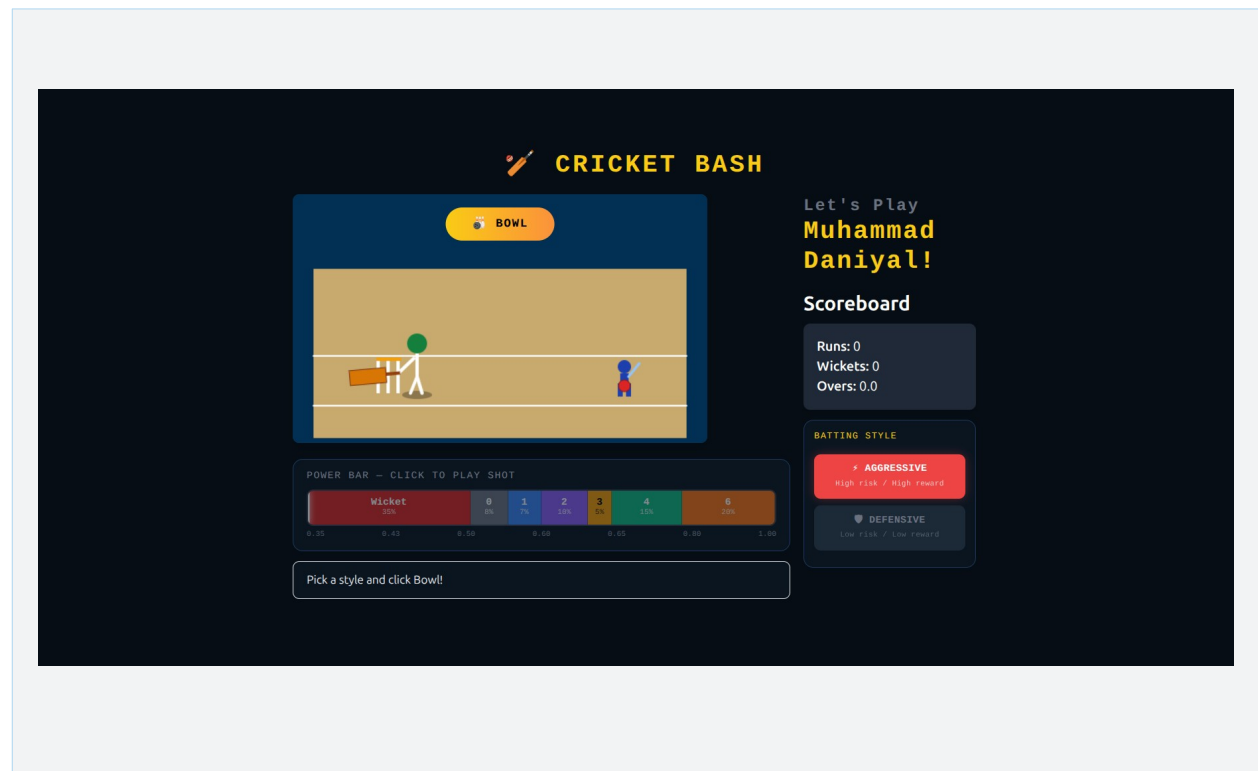


Figure 1: Player using Aggressive batting style. Note the high boundary/six probability segments visible in the power bar.

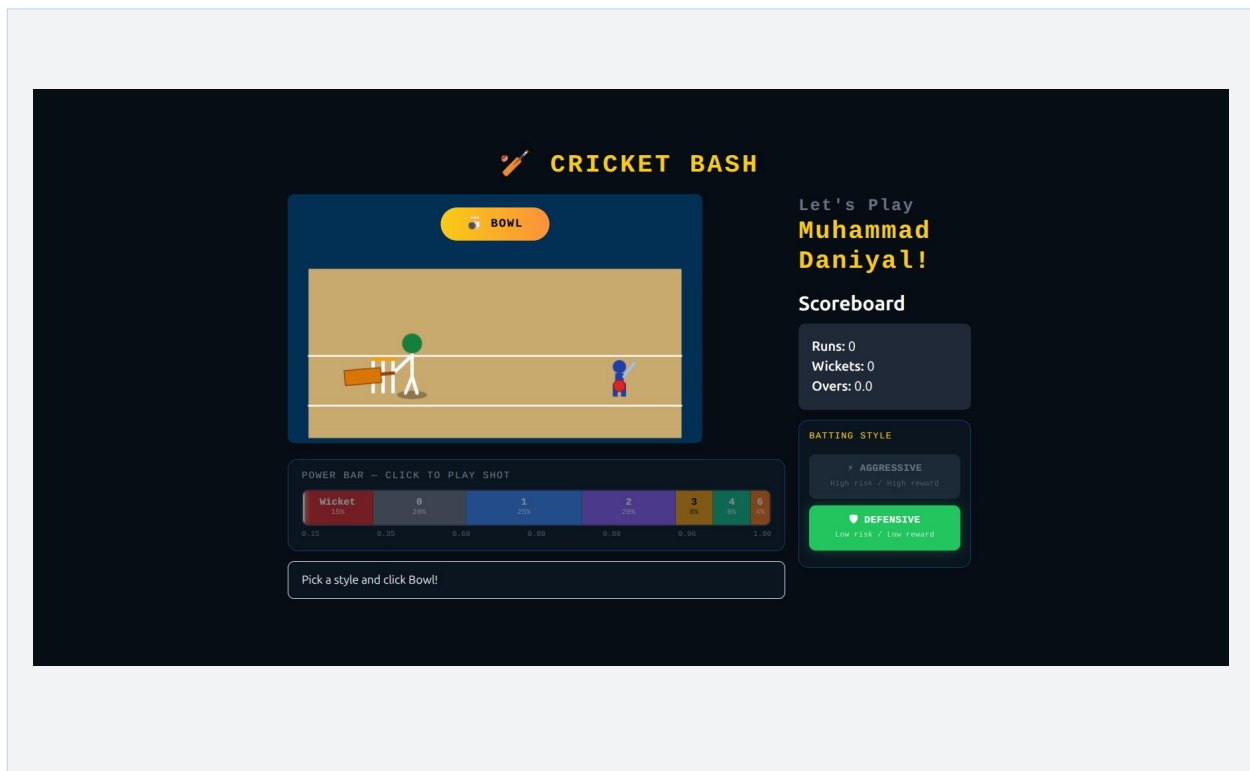
Figure 2 — Defensive Batting in Action

Figure 2: Player using Defensive batting style. The power bar shows a dominant safe-zone segment (0–2 runs).

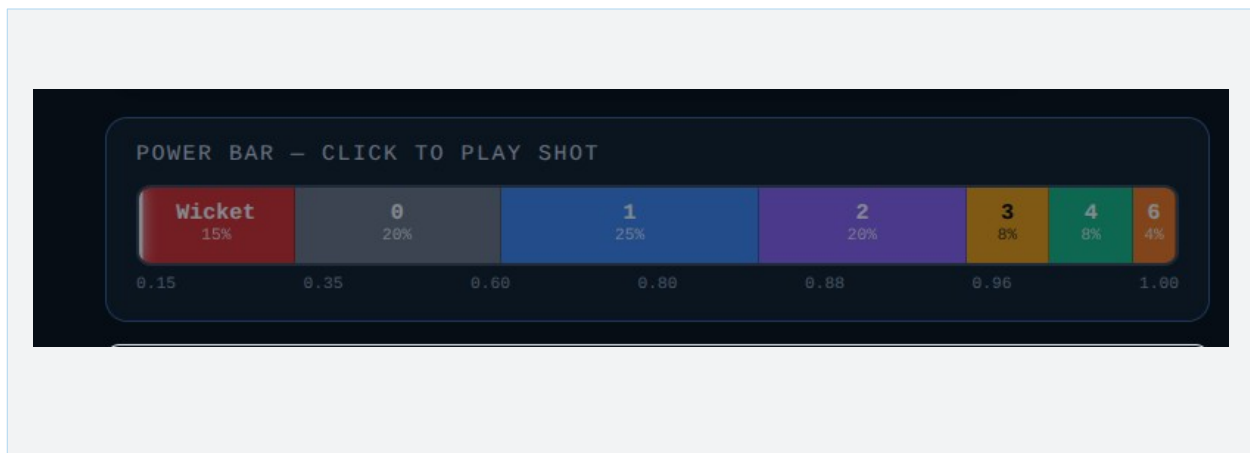
Figure 3 — Power Bar Visible During Gameplay

Figure 3: The active power bar with the oscillating slider and colour-coded outcome segments.

Figure 4 — Scoreboard Updates



Figure 4: Live scoreboard reflecting runs, wickets, overs, and ball-by-ball commentary after several deliveries.

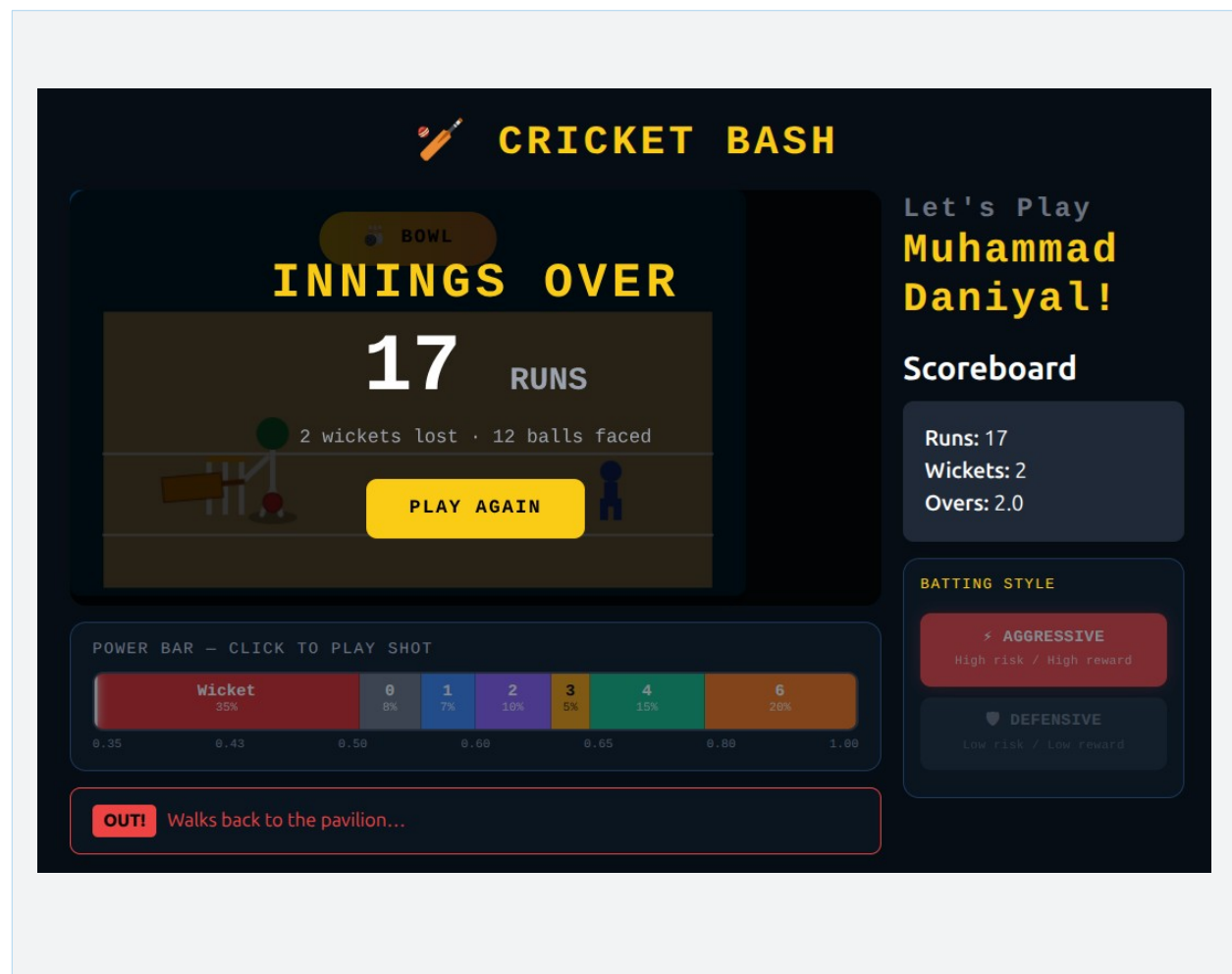
Figure 5 — Game Over Screen

Figure 5: The Game Over canvas overlay displayed at innings completion, showing the final score summary.

7. Conclusion

Cricket Bash successfully demonstrates the application of modern front-end technologies to build an engaging, interactive browser game. The project highlights several key web programming competencies:

- Reactive state management — Zustand provides a clean, minimal-boilerplate solution for coordinating game phases across multiple components.
- Canvas-based animation — `requestAnimationFrame` with delta-time scaling produces smooth, hardware-independent animations for bowling, shot trajectories, and the power bar.

- Probabilistic game design — mapping cumulative probability segments to a positional input (the power bar slider) creates a skill-influenced, yet unpredictable outcome model that authentically simulates batting risk.
- Dual gameplay modes — the Aggressive vs. Defensive dichotomy adds meaningful strategic depth, requiring the player to consciously balance run rate against wicket preservation.

The project achieves its core objective of creating a playable, well-structured web application that integrates game logic, animation, and state management within a single-page React architecture.